Online Food Ordering System

Problem Statement

Develop a database for an Online Food Ordering System that enables customers to browse restaurant menus, place food orders, and make payments. The system should efficiently manage restaurant information, menu items, customer orders, and payment details while generating sales and order reports.

Expected Outcome

The system should be able to:

- Maintain restaurant information.
- Store menu details.
- Manage customer orders.
- Track payment information.
- Generate restaurant sales reports.
- Display customer order history.
- Identify popular food items.

Database Design

- **Restaurants** (restaurant_id PK, restaurant_name, location, cuisine)
- **Menu** (menu_id PK, restaurant_id FK, item_name, price, availability)
- **Customers** (customer_id PK, customer_name, phone, city)
- **Orders** (order_id PK, customer_id FK, menu_id FK, quantity, order_date, order_status)
- **Payments** (payment_id PK, order_id FK, payment_method, amount, payment_status, payment_date)

Sample Data

Restaurants (5 Records)

INSERT INTO Restaurants VALUES

(101,'Spicy Kitchen', 'Hyderabad', 'Indian'),
(102,'Pizza Hub', 'Bengaluru', 'Italian'),
(103,'Dragon Bowl', 'Chennai', 'Chinese'),
(104,'Burger World', 'Vijayawada', 'Fast Food'),
(105,'Ocean Grill', 'Visakhapatnam', 'Seafood');

Menu (10 Records)

INSERT INTO Categories VALUES

(201,101,'Chicken Biryani',250,'Available'), (202,101,'Paneer Curry',180,'Available'),
(203,102,'Veg Pizza',350,'Available'), (204,102,'Cheese Pasta',280,'Available'),
(205,103,'Noodles',220,'Available'), (206,103,'Fried Rice',240,'Unavailable'), (207,104,'Chicken Burger',180,'Available'), (208,104,'French Fries',120,'Available'), (209,105,'Grilled Fish',450,'Available'), (210,105,'Prawn Curry',500,'Available');

Customers (10 Records)

INSERT INTO Customers VALUES

(301,'Aarav','9876500001','Hyderabad'), (302,'Bhavya','9876500002','Vijayawada'),
(303,'Charan','9876500003','Vizag'), (304,'Divya','9876500004','Bengaluru'),
(305,'Esha','9876500005','Chennai'), (306,'Farhan','9876500006','Mumbai'),
(307,'Gopi','9876500007','Warangal'), (308,'Harini','9876500008','Guntur'),
(309,'Ishaan','9876500009','Pune'), (310,'John','9876500010','Kochi');

Orders (12 Records)

INSERT INTO Orders VALUES

(401,301,201,2,'2026-07-01','Delivered'), (402,302,203,1,'2026-07-02','Delivered'),
(403,303,205,3,'2026-07-03','Preparing'), (404,304,207,2,'2026-07-04','Delivered'),
(405,305,210,1,'2026-07-05','Cancelled'), (406,306,202,2,'2026-07-06','Delivered'),
(407,307,208,4,'2026-07-07','Delivered'), (408,308,209,1,'2026-07-08','Preparing'),
(409,309,204,2,'2026-07-09','Delivered'), (410,310,201,1,'2026-07-10','Delivered'),
(411,301,210,2,'2026-07-11','Preparing'), (412,303,203,1,'2026-07-12','Delivered');

Payments (12 Records)

INSERT INTO Payments VALUES

(501,401,'UPI',500,'Paid','2026-07-01'),
(502,402,'Card',350,'Paid','2026-07-02'),
(503,403,'UPI',660,'Paid','2026-07-03'),
(504,404,'Cash',360,'Paid','2026-07-04'),
(505,405,'Card',500,'Refunded','2026-07-05'),
(506,406,'UPI',360,'Paid','2026-07-06'),
(507,407,'Cash',480,'Paid','2026-07-07'),
(508,408,'UPI',450,'Pending','2026-07-08'),
(509,409,'Card',560,'Paid','2026-07-09'),
(510,410,'UPI',250,'Paid','2026-07-10'),
(511,411,'Card',1000,'Pending','2026-07-11'),
(512,412,'Cash',350,'Paid','2026-07-12');

SQL Questions

1. Display all restaurants and their cuisine types.
2. Update the order status from **Preparing** to **Delivered**.
3. Increase the price of all food items in a specific restaurant by 10%.
4. Delete a cancelled order.
5. Display all restaurant names in uppercase
6. Display all menu item names in lowercase.
7. Display the first five characters of each menu item.
8. Concatenate the customer name and city.
9. Replace the word **Chicken** with **Grilled** in product names.
10. Display last week orders.
11. Display the day name for every order date.
12. Find the number of days since each order was placed.
13. Display the payment date in **DD-Monday-YYYY** format.
14. Display the month and year of each payment.
15. Count the total number of menu items available in each restaurant.
16. Display the average product of menu items for every restaurant.
17. Find the highest payment amount received.
18. Display the total revenue generated by each restaurant.
19. Find the total quantity ordered for each menu item.
20. Display customer names along with the food items they ordered.
21. Display restaurant names with their menu items.
22. Display customer names, ordered food items, and order status.
23. Display restaurant names, menu items, quantity ordered, and payment amount.
24. Display all restaurants and their menu items, even if no orders have been placed
25. Display customer names, restaurant names, ordered food items, and order dates in a single result.
26. Find customers whose payment amount is greater than the average payment amount.
27. Display the restaurant that serves the most expensive food item.
28. Find customers who ordered the highest-priced menu item.
29. Display restaurants that have **never received an order**
30. Create a view named **Customer_Order_History** displaying customer name, restaurant name, menu item, quantity, and order status.
31. Create a view named **Restaurant_Revenue** displaying restaurant name, total orders, and total revenue.
32. Retrieve records from both views.
33. Start a transaction. Update theprice of a menu item. Create a SAVEPOINT. Then update the payment status of an order. Roll back to the savepoint. Commit the transaction.
34. Create a stored procedure to display all orders placed by a particular customer.
    - Input: customer_id
    - Display: order_id, menu item, quantity, order date, and order status.

35. Create a stored procedure to calculate the total amount spent by a particular customer.
    - Input: customer_id
    - Use the Payments table and display the total payment amount
36. Create a trigger on the Orders table.
    Whenever an order is deleted, automatically store the deleted order details in an Orders_Archive table.
37. Create a trigger on the Orders table.
    Whenever the order_status changes, automatically store the old status, new status, order_id, and change date in an Order_Status_History table.

    .